Lab 5

Lab 4 gave the game interactive objects — things that update, render, and eventually remove themselves. But those objects floated on a blank screen. Lab 5 introduces a tile-based terrain system: the game now loads a map created in the Tiled editor, parses its JSON format, resolves tile sets, and renders a grid of small grass tiles as the background behind everything else.

Assets — a Tiled map and its tile set

The commit moves `BombExploding.png` into a new `Assets/` folder and adds five new files there: four 16×16 grass tile images (`grass_001.png` through `grass_004.png`), a tile set definition (`grass.tsj`), and a map definition (`terrain.tmj`).

The `grass.tsj` and `terrain.tmj` definition files are created using a 2D sprite editor called Tiled.

`grass.tsj` is a Tiled tile set file in JSON format. It declares four tiles, each pointing at one of the grass images:

```
{
  "name": "grass",
  "tilecount": 4,
  "tilewidth": 16,
  "tileheight": 16,
  "tiles": [
    { "id": 0, "image": "grass_001.png", "imageheight": 16, "imagewidth": 16 },
    { "id": 1, "image": "grass_002.png", "imageheight": 16, "imagewidth": 16 },
    { "id": 2, "image": "grass_003.png", "imageheight": 16, "imagewidth": 16 },
    { "id": 3, "image": "grass_004.png", "imageheight": 16, "imagewidth": 16 }
  ]
}
```

Each tile has an `id` (0–3), a reference to its image file, and the image dimensions. The tile set itself records the common tile size (16×16) and the total count.

`terrain.tmj` is a Tiled map file. It defines a 60×40 grid of 16×16 tiles — giving a 960×640 pixel playing field. It contains a single layer ("Tile Layer 1") with a flat `data` array of 2400 integers (60 × 40). Each integer is a 1-based tile ID referencing the grass tile set. The map also contains a `tilesets` array that points to `grass.tsj` by filename with a `firstgid` of 1:

```
{
  "height": 40,
  "width": 60,
  "tileheight": 16,
  "tilewidth": 16,
  "layers": [
    {
      "data": [2, 2, 1, 1, 1, 1, 3, 1, ...],
      "height": 40,
```

```
      "width": 60,
      "name": "Tile Layer 1",
      "type": "tilelayer"
    }
  ],
  "tilesets": [
    { "firstgid": 1, "source": "grass.tsj" }
  ]
}
```

The `firstgid` field is important — Tiled uses 1-based IDs in the data array, so a value of `1` in the data corresponds to tile ID `0` in the tile set. This offset is handled during rendering.

Data models — deserializing Tiled JSON into C# objects

Four new classes under `Models/Data/` mirror the JSON structures so that `System.Text.Json` can deserialize them directly.

`Models/Data/Level.cs` represents the top-level map file. It captures the grid dimensions, the list of layers, and the tile set references:

```
public class Level
{
    [JsonPropertyName("height")]
    public int? Height { get; set; }

    [JsonPropertyName("width")]
    public int? Width { get; set; }

    [JsonPropertyName("layers")]
    public List<Layer> Layers { get; set; } = new();

    [JsonPropertyName("tilesets")]
    public List<TileSetReference> TileSets { get; set; } = new();

    [JsonPropertyName("tileheight")]
    public int? TileHeight { get; set; }

    [JsonPropertyName("tilewidth")]
    public int? TileWidth { get; set; }

    // ... orientation, renderorder, version, etc.
}
```

`Models/Data/Layer.cs` represents a single tile layer. The key property is `Data` — the flat array of tile IDs that defines which tile goes in each cell of the grid:

```
public class Layer
{
```

```
    [JsonPropertyName("data")]
    public List<int?> Data { get; set; } = new();

    [JsonPropertyName("height")]
    public int? Height { get; set; }

    [JsonPropertyName("width")]
    public int? Width { get; set; }

    [JsonPropertyName("name")]
    public string Name { get; set; } = "";

    // ... id, opacity, visible, x, y, type
}
```

`Models/Data/TileSet.cs` holds the tile set definition (the parsed `grass.tsj`). It also defines `TileSetReference`, a small class for the entries in the map's `tilesets` array that point to external `.tsj` files:

```
public class TileSet
{
    [JsonPropertyName("name")]
    public string Name { get; set; } = "";

    [JsonPropertyName("tilecount")]
    public int? TileCount { get; set; }

    [JsonPropertyName("tiles")]
    public List<Tile> Tiles { get; set; } = new();

    [JsonPropertyName("tilewidth")]
    public int? TileWidth { get; set; }

    [JsonPropertyName("tileheight")]
    public int? TileHeight { get; set; }
}

public class TileSetReference
{
    [JsonPropertyName("firstgid")]
    public int? FirstGID { get; set; }

    [JsonPropertyName("source")]
    public string Source { get; set; } = "";
}
```

`Models/Data/Tile.cs` represents a single tile within a tile set. It stores the tile's ID, its image path, and the image dimensions. Crucially, it also has a `TextureId` property marked `[JsonIgnore]` — this is not part of the JSON but is assigned at runtime after the tile's image has been loaded into the GPU:

```
public class Tile
{
    [JsonPropertyName("id")]
    public int? Id { get; set; }

    [JsonPropertyName("image")]
    public string Image { get; set; } = "";

    [JsonPropertyName("imageheight")]
    public int? ImageHeight { get; set; }

    [JsonPropertyName("imagewidth")]
    public int? ImageWidth { get; set; }

    [JsonIgnore]
    public int TextureId { get; set; } = -1;
}
```

The `[JsonIgnore]` attribute prevents the serializer from trying to read or write this property. It exists purely as a bridge between the data layer and the rendering layer — once a tile's image is loaded as an SDL texture, the resulting texture handle is stashed here so the renderer can find it later.

GameLogic — loading and rendering the level

`GameLogic` gains three new fields to hold the parsed level data:

```
private readonly Dictionary<string, TileSet> _loadedTileSets = new();
private readonly Dictionary<int, Tile> _tileIdMap = new();
private Level _currentLevel = new();
```

`_loadedTileSets` stores tile sets by name, `_tileIdMap` provides direct tile lookup by ID, and `_currentLevel` holds the deserialized map.

The `InitializeGame` method, which was previously empty, now does the heavy lifting of loading the level. It reads the map JSON, deserializes it, then follows each tile set reference to load and deserialize the tile set files. For every tile in every tile set, it loads the tile's image as an SDL texture and records the resulting texture handle:

```
public void InitializeGame()
{
    var levelContent = File.ReadAllText(Path.Combine("Assets", "terrain.tmj"));
    var level = JsonSerializer.Deserialize<Level>(levelContent);
    if (level == null)
    {
        throw new Exception("Failed to load level");
    }
```

```
    foreach (var tileSetRef in level.TileSets)
    {
        var tileSetContent = File.ReadAllText(Path.Combine("Assets",
tileSetRef.Source));
        var tileSet = JsonSerializer.Deserialize<TileSet>(tileSetContent);
        if (tileSet == null)
        {
            throw new Exception("Failed to load tile set");
        }

        foreach (var tile in tileSet.Tiles)
        {
            tile.TextureId = GameRenderer.LoadTexture(Path.Combine("Assets",
tile.Image), out _);
            _tileIdMap.Add(tile.Id!.Value, tile);
        }

        _loadedTileSets.Add(tileSet.Name, tileSet);
    }

    _currentLevel = level;
}
```

The loading follows a two-step resolution pattern: the map references tile sets by filename, and each tile set references individual tile images by filename. By the end of `InitializeGame`, every tile image is loaded into GPU memory and its handle is stored in the `Tile` object, ready for rendering.

The new `RenderTerrain` method iterates over every layer in the level and draws each cell. It loops column-by-column (`i`) and row-by-row (`j`), computes the index into the flat data array, subtracts 1 to convert from Tiled's 1-based IDs to the 0-based tile IDs, looks up the tile, and draws it at the correct pixel position:

```
public void RenderTerrain(GameRenderer renderer)
{
    foreach (var currentLayer in _currentLevel.Layers)
    {
        for (int i = 0; i < _currentLevel.Width; ++i)
        {
            for (int j = 0; j < _currentLevel.Height; ++j)
            {
                int? dataIndex = j * currentLayer.Width + i;
                if (dataIndex == null)
                {
                    continue;
                }

                var currentTileId = currentLayer.Data[dataIndex.Value] - 1;
                if (currentTileId == null)
                {
                    continue;
                }
```

```
                var currentTile = _tileIdMap[currentTileId.Value];

                var tileWidth = currentTile.ImageWidth ?? 0;
                var tileHeight = currentTile.ImageHeight ?? 0;

                var sourceRect = new Rectangle<int>(0, 0, tileWidth,
tileHeight);
                var destRect = new Rectangle<int>(i * tileWidth, j *
tileHeight, tileWidth, tileHeight);
                renderer.RenderTexture(currentTile.TextureId, sourceRect,
destRect);
            }
        }
    }
}
```

The formula `j * currentLayer.Width + i` converts a 2D grid position into an index in the flat data array — standard row-major indexing. Each tile is drawn at `(i * tileWidth, j * tileHeight)`, tiling the entire map surface without gaps.

The `- 1` on the tile ID deserves attention: Tiled stores tile IDs starting at `firstgid` (which is 1), but the tile set's internal IDs start at 0. Subtracting 1 bridges that gap.

GameRenderer — a new RenderTexture method and render ordering

The renderer gains a general-purpose `RenderTexture` method that takes a texture ID and source/destination rectangles:

```
public void RenderTexture(int textureId, Rectangle<int> src, Rectangle<int>
dst)
{
    if (_texturePointers.TryGetValue(textureId, out var texture))
    {
        _sdl.RenderCopy((Renderer*)_renderer, (Texture*)texture, in src, in
dst);
    }
}
```

This is a lower-level rendering primitive compared to `RenderGameObject`. Where `RenderGameObject` takes an entire `RenderableGameObject` and pulls the rectangles from its properties, `RenderTexture` works with raw texture handles and rectangles. This makes it suitable for tile rendering, where there are no game objects — just texture IDs and grid positions.

The render loop in the `Render` method now calls terrain rendering before object rendering:

```
_gameLogic.RenderTerrain(this);
_gameLogic.RenderAllObjects(timeSinceLastFrame, this);
```

The order matters. Terrain is drawn first, creating the background layer. Game objects (like bomb explosions) are drawn on top of it. This is a painter's algorithm approach — things drawn later appear in front of things drawn earlier.

Project file — wildcard asset copying

The `.csproj` file changes from copying a single named file to using a wildcard pattern:

```
<Content Include="Assets\**\*">
  <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
</Content>
```

Previously, only `BombExploding.png` was explicitly listed. Now, everything under `Assets/` — the tile images, the JSON map and tile set files, and the bomb sprite sheet — is automatically copied to the output directory during build. This means new assets can be added to the folder without touching the project file again.

The bomb path fix

A small but necessary change: `AddBomb` now prepends `"Assets"` to the bomb sprite path, since the image was moved into the `Assets/` folder:

```
AnimatedGameObject bomb = new AnimatedGameObject(
    Path.Combine("Assets", "BombExploding.png"), 2, _bombIds, 13, 13, 1, x, y);
```

Without this change, the bomb animation would have broken because the file is no longer at the project root.

Summary

By the end of Lab 5, the game has a visible world. Instead of objects floating on a black screen, there is a 960×640 grass terrain built from four 16×16 tile variants arranged in a 60×40 grid. The level data comes from the Tiled map editor's JSON format, parsed through a set of C# data models that mirror the JSON structure. The terrain renders first each frame, and game objects (bomb explosions) render on top of it. The foundation is now in place for building more complex maps with multiple tile sets and layers.